MILESTONE 2 REPORT

Data.gov Web Crawler & Database Population

Author	Salma Elmarakby
Student ID	900232658
Course	Database Systems
Milestone	2 – Web Crawler & Database
Deadline	31 March 2026
Data Source	data.gov CKAN API
Datasets	2,000 (2 pages × 1,000 rows)
Database	MySQL — `data_gov_db`

Contents

1 Introduction **2**

2 System Architecture **2**

3 Database Schema **3**

4 Generated CSV Files **3**

5 User Tables & Random Usage Generation **4**
5.1 User Table Population 4
5.2 Random Usage Generation 4

6 Streamlit Dashboard **5**

7 Execution Instructions **6**
7.1 Prerequisites 6
7.2 Run the Full Pipeline 6
7.3 Run Individual Stages 6

8 Conclusion **6**

1. Introduction

This report documents the design, implementation, and results of the Milestone 2 deliverable for the Database Systems course. The primary objective was to build a fully automated data pipeline that:

- Crawls and fetches dataset metadata from the **data.gov** catalog
- Parses and extracts all fields required to populate the non-user database tables
- Populates a redesigned MySQL relational schema (`data_gov_db`)
- Generates structured CSV exports for every table
- Produces 500 randomly generated, reproducible usage records for the user-activity table

2. System Architecture

The pipeline is divided into three sequential scripts, orchestrated by a single entry point (`run_pipeline.py`).

Table 1: Pipeline scripts and their responsibilities

Script	Stage	Responsibility
`Crawler.py`	Stage 1 & 2	API fetch → field extraction → write all CSVs
`populate_database_redesigned.py`	Stage 3	Read CSVs → create MySQL schema → insert rows
`dashboard.py`	Visualisation	Streamlit interactive dashboard over raw CSV data
`run_pipeline.py`	Orchestrator	Calls all scripts in order; validates output file existence

The overall data flow is:

```
data.gov CKAN API ⟶ JSON parse ⟶ CSV files
⟶ MySQL population ⟶ Streamlit dashboard
```

3. Database Schema

Table 2: Relational Model — all tables, primary keys, and foreign keys

Table Name	Attributes (Primary Keys [PK], Foreign Keys [FK])
Organization	*OrganizationID* ***[PK]***, OrganizationName, OrganizationType, ContactInformation, Description
User	*Email* ***[PK]***, Username, Gender, BirthDate, Country
Dataset	*DatasetID* ***[PK]***, DatasetName, Description, AccessLevel, License, MetadataCreateDate, MetadataUpdateDate, Maintainer, *OrganizationID* ***[FK]***
Publisher	*DatasetID* ***[PK][FK]***, *PublisherName* ***[PK]***
Tag	*TagName* ***[PK]***
Topic	*TopicName* ***[PK]***, Category
Resources	*URL* ***[PK]***, Format, *DatasetID* ***[FK]*** (Required to link to Dataset)
Dataset_Tags	*DatasetID* ***[PK][FK]***, *TagID* ***[PK][FK]***
Dataset_Topics	*DatasetID* ***[PK][FK]***, *TopicName* ***[PK][FK]***
Dataset_user	*Email* ***[PK][FK]***, *DatasetID* ***[PK][FK]***, *ProjectName* ***[PK]***, ProjectCategory

4. Generated CSV Files

Seven CSV files are produced in a structured `output/` directory tree and serve as the single source of truth for database loading:

Table 3: Output CSV files

File Path	Rows	Description
`output/raw_data/crawled_datasets_api.csv`	2,000	Full raw dataset metadata from API
`output/reference/organizations.csv`	varies	Unique organisations with assigned IDs
`output/reference/tags.csv`	varies	Unique file-format tags (lowercased)
`output/datasets/datasets.csv`	2,000	Dataset records with FK to Organisation
`output/datasets/dataset_tags.csv`	varies	Many-to-many dataset ↔ tag mappings
`output/users/users_final.csv`	from `users.cs`	Processed user records
`output/users/usage.csv`	500	Randomly generated usage entries

5. User Tables & Random Usage Generation

5.1 User Table Population

The `User` table is populated directly from the provided `users.csv` file (located at `../users.csv` relative to the scripts directory). Each row is mapped as follows:

- `email` → primary key `Email`
- `username` → `Username`
- `gender` → `Gender`
- `age` → `BirthDate` (approximated: `now() - age` × `365 days`)
- `country` → `Country`

5.2 Random Usage Generation

500 usage records are generated with `numpy.random.seed(42)` set for full reproducibility. Each record randomly selects:

- A `user_id` from $[1, N_{\text{users}}]$
- A `dataset_id` from $[1, N_{\text{datasets}}]$
- An `access_count` $\in [1, 100]$ and `download_count` $\in [0, \text{access_count}]$
- A `last_accessed` date within the past 180 days from execution time

- A `usage_type` drawn from {`view`, `download`, `analyze`, `share`}

6. Streamlit Dashboard

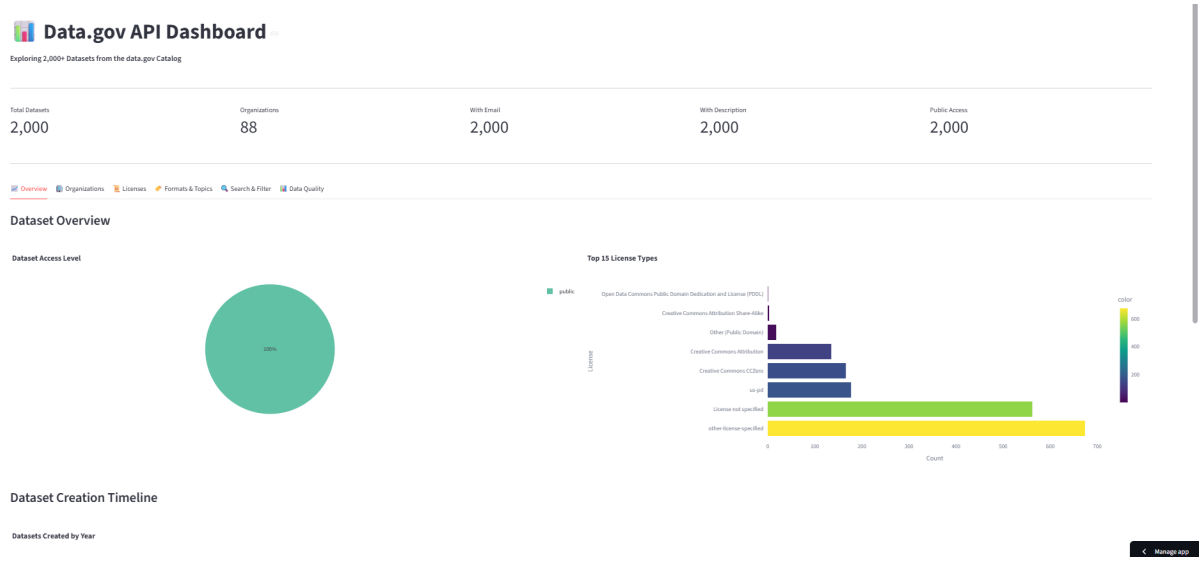

Figure 1: Dashboard interface

The dashboard is **live and publicly deployed** at:

https://crawler-4zztb6gnqzgckaslxwqhb2.streamlit.app/

`dashboard.py` implements a six-tab interactive dashboard powered by **Streamlit** and **Plotly**:

Table 4: Dashboard tabs and their contents

Tab	Content
Overview	Access-level pie chart, top-15 licence bar chart, dataset creation timeline
Organizations	Top-20 organisations ranked bar chart + key metrics panel
Licenses	Pie chart of all licence types; progress-bar breakdown of top 10
Formats & Topics	Top-15 file formats; top-15 topic/tag bar charts
Search & Filter	Full-text name search + dropdown filters by organisation and licence
Data Quality	Completeness % per field + auto-detected data issues

To run locally instead:

```
streamlit run dashboard.py
```

The dashboard also provides one-click CSV and plain-text report downloads via Streamlit's `st.download_button`.

7. Execution Instructions

7.1 Prerequisites

- Python 3.9+
- MySQL 8.x running on `127.0.0.1:3306`
- Required Python packages (install once):

```
pip install requests pandas numpy mysql-connector-python streamlit plotly
```

7.2 Run the Full Pipeline

```
python run_pipeline.py
```

This single command executes all three stages (API crawl → CSV generation → MySQL population) and validates that every output file was created successfully before proceeding to the next stage.

7.3 Run Individual Stages

```
python complete_pipeline.py
python populate_database_redesigned.py
streamlit run dashboard.py
```

8. Conclusion

The Milestone 2 deliverable successfully implements an end-to-end data pipeline that fetches 2,000 dataset records from the data.gov CKAN API, normalises the data into a relational MySQL schema spanning 10 tables, and exports every table as a CSV file. The API-based approach is a pragmatic and defensible choice over HTML scraping: it is faster, more stable under site changes, and yields higher-quality structured data with no rendering overhead.